UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación, Sede Sololá.
Ingeniero Wilmer Orlando Xeron
Base de Datos 1 I Sec. “A”



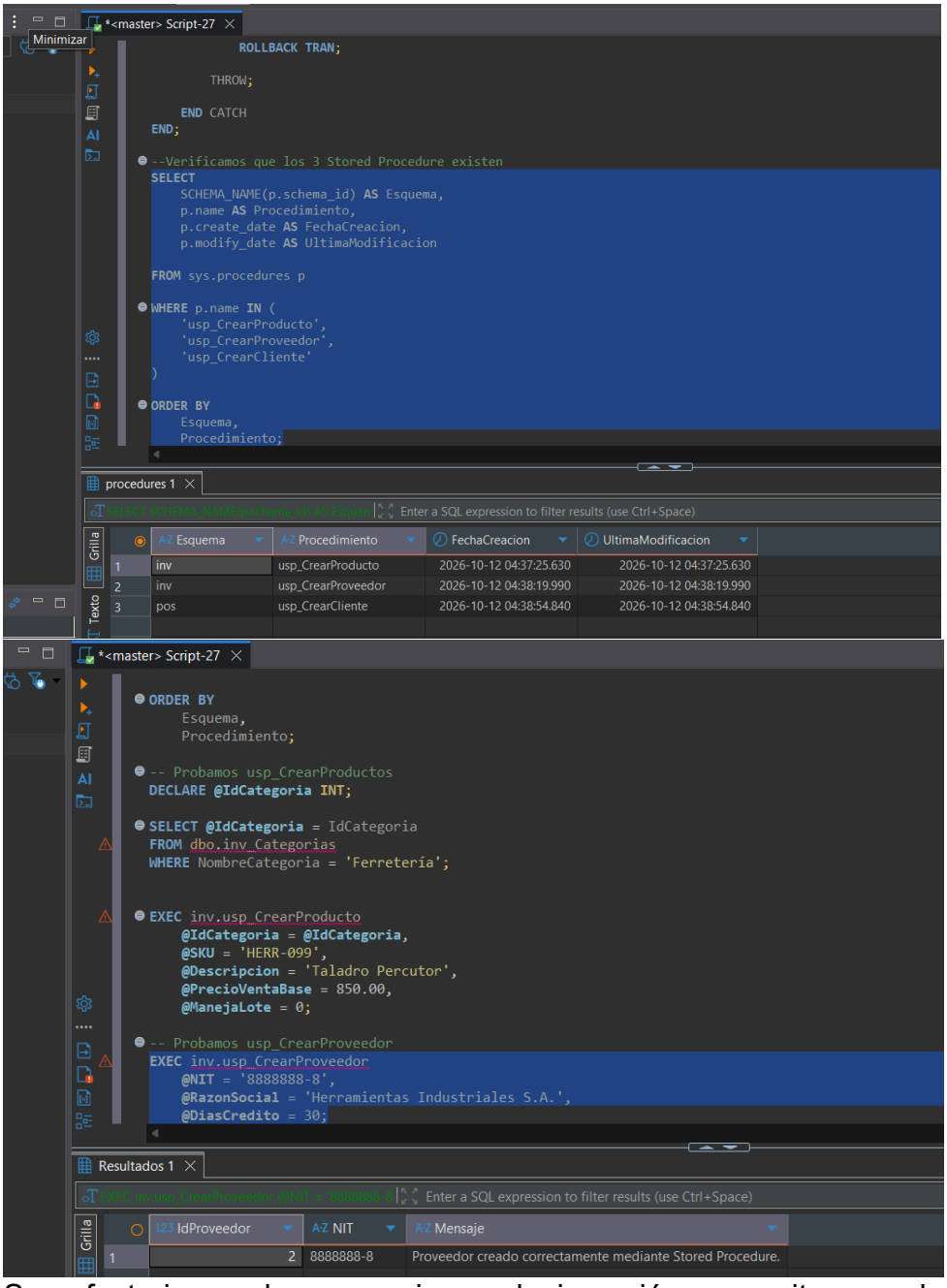
Fase 4: PVISTAS INDEXADAS, CONEXIÓN CON PYTHON STREAMLIT

Video: Sustentación
GitHub: Cristhian

Cristhian Cesar Felipe Mucún Queché
2290-17-17851

El gran Cierre (Frontend, Vistas Materializadas y Defensa)

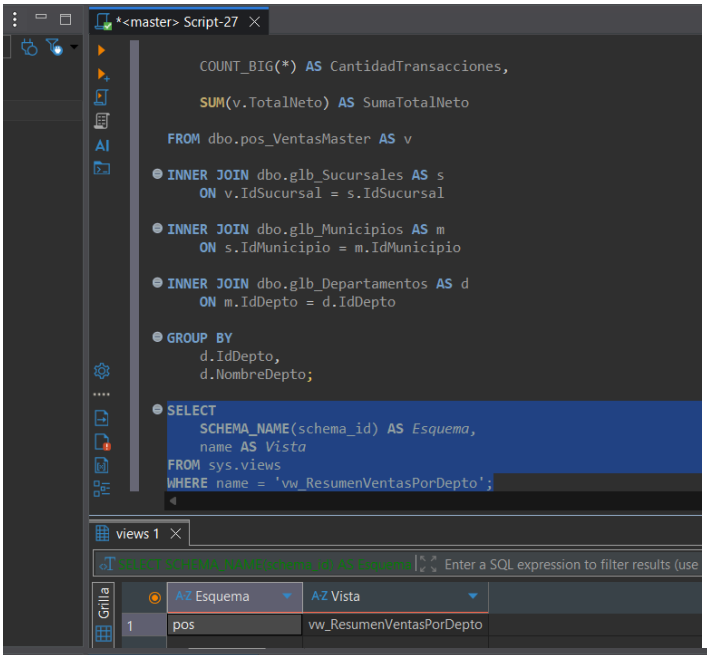
- Refactorización a Procedimientos Almacenados:

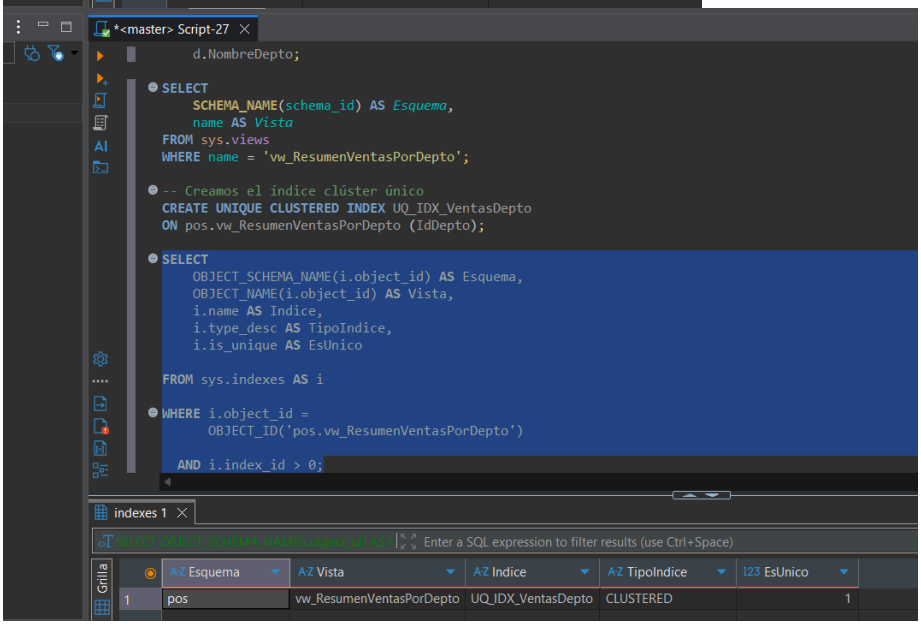


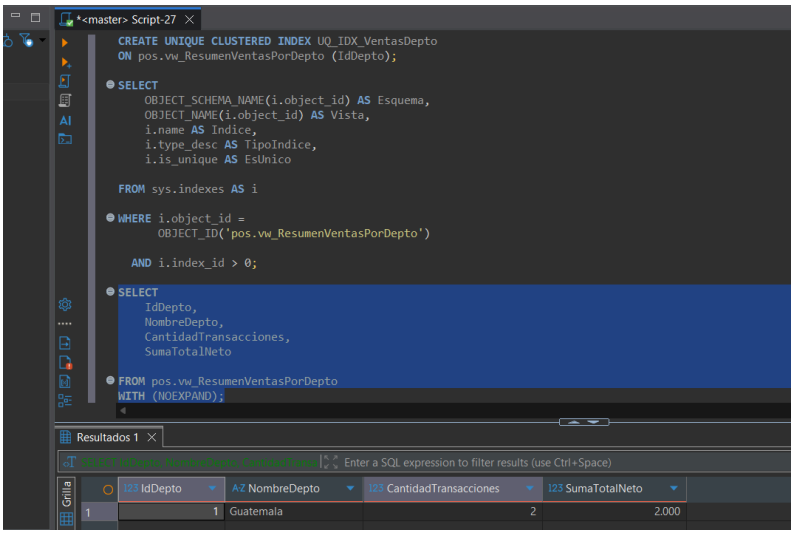
Se refactorizaron las operaciones de inserción par evitar que las aplicaciones ejecuten DML directamente sobre las tablas principales. Se crearon procedimientos almacenados para productos, proveedores y clientes, cumpliendo el nuevo estándar de encapsulación solicitado para al menos tres catálogos. Cada procedimiento recibe únicamente los parámetros necesarios, valida las reglas empresariales y realiza la inserción dentro de una transacción controlada mediante TRY/CATCH,

COMMIT, ROLLBACK, de esta manera, las aplicaciones consumidoras no necesitan conocer directamente la estructura completa de las tablas y se reduce el riesgo de insertar información inconsistente.
El procedimiento de productos proporcionado como ejemplo únicamente comprobaba que el precio fuera mayor a cero antes de insertar. Se amplió la validación para verificar la existencia de la categoría, impedir SKU duplicados y evitar valores vacíos en SKU y descripción. Además se incorporó SET XACT_ABORT ON y SCOPE_IDENTITY() para obtener de forma segura el identificador generado.

- Vistas Indexadas

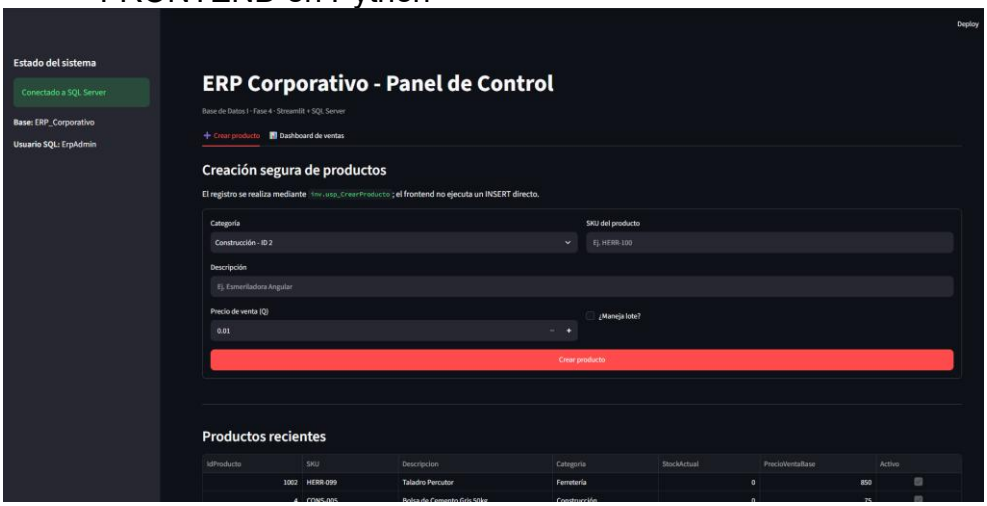






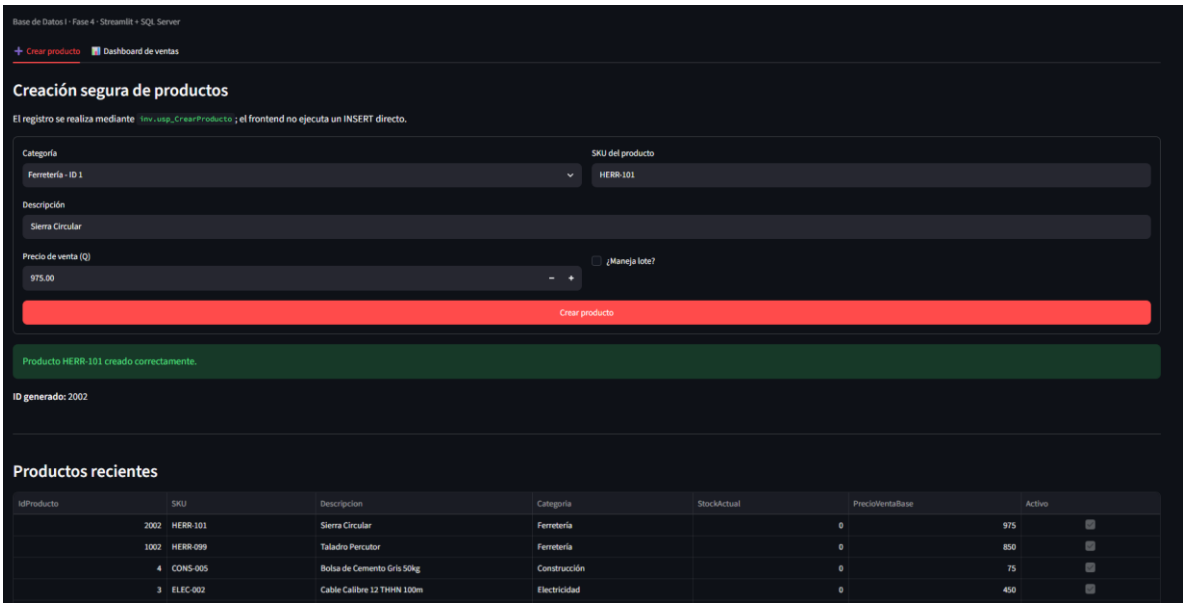
Este bloque crea una vista indexada destinada al dashboard del ERP. La vista relaciona las ventas con sucursales, municipios y departamentos para calcular la cantidad de transacciones y el total neto vendido por departamento. Se utiliza SCHEMABINDING para vincular la definición de la vista con la estructura de sus tablas base y COUNT_BIG(*), requerido por SQL Server para este tipo de agregaciones en vistas indexadas. Posteriormente se crea un índice clúster único, provocando que el resultado agregado sea almacenado físicamente y pueda consultarse de manera más eficiente para reportes repetitivos.
El código proporcionado hacía referencia a objetos como pos.VentasMaster y glb.Sucursales, aunque las tablas creadas realmente durante la Fase 1 quedaron bajo nombres como dbo.pos_VentasMaster y dbo.glb_Sucursales. Se corrigieron las referencias para utilizar los objetos reales del proyecto. También se agregó IdDepto al resultado y al GROUP BY, utilizando este identificador como clave del índice único en lugar del nombre textual del departamento, evitando depender de nombres potencialmente repetidos o modificables.

- FRONTEND en Python

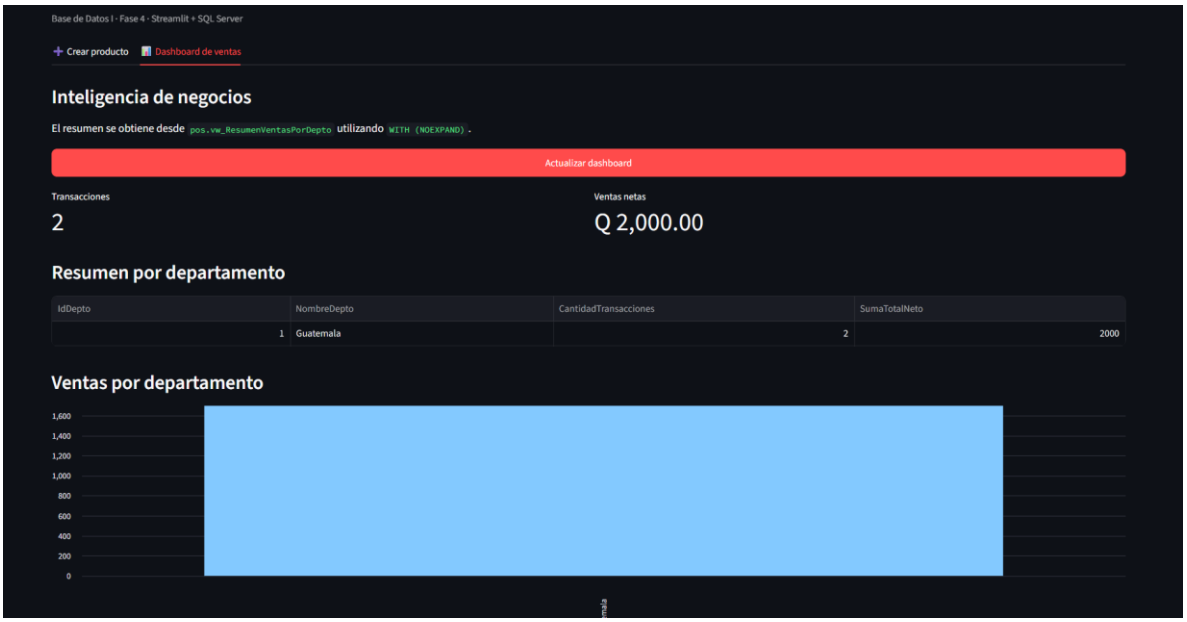


La aplicación web desarrollada en Streamlit se encuentra conectada correctamente a la base de datos ERP_Corporativo utilizando el usuario ErpAdmin. La interfaz

obtiene dinámicamente las categorías y los productos almacenados en SQL Server, demostrando la comunicación entre el contenedor de Streamlit y el contenedor de SQL Server mediante la red interna de Docker. Además, el formulario de creación de productos no realiza inserciones directas sobre la tabla, sino que utiliza el procedimiento almacenado inv.usp_CrearProducto, manteniendo las validaciones y reglas de negocio dentro del motor de base de datos.



En esta prueba se registró el producto HERR-101 - Sierra Circular desde la interfaz Streamlit. El frontend no ejecutó un INSERT directo sobre inv_Productos, sino que llamó al procedimiento almacenado inv.usp_CrearProducto, el cual validó la categoría, el SKU, la descripción y el precio antes de confirmar la transacción. SQL Server devolvió el identificador generado y, posteriormente, el producto apareció en la sección de productos recientes, demostrando que la información quedó persistida correctamente en ERP_Corporativo. Este flujo mantiene las reglas de negocio centralizadas en el motor y permite que Streamlit funcione únicamente como capa de presentación.



El dashboard de inteligencia de negocios consume la vista indexada pos.vw_ResumenVentasPorDepto utilizando WITH (NOEXPAND), permitiendo consultar directamente el resumen materializado de ventas por departamento. La prueba muestra dos transacciones correspondientes al departamento de Guatemala con un total neto acumulado de Q2,000.00, valor que coincide con las ventas registradas previamente por Q500.00 y Q1,500.00. Streamlit recibe estos datos desde SQL Server y los presenta mediante indicadores, una tabla y una gráfica, manteniendo la lógica de agregación dentro del motor de base de datos y utilizando el frontend únicamente como capa de visualización.